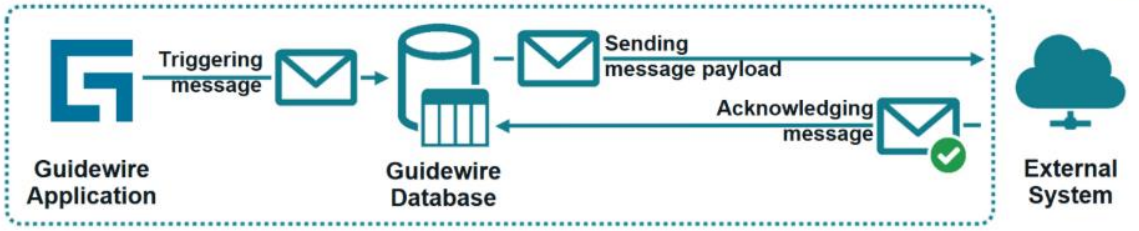
Messaging Architecture

Messaging Stages

What is Guidewire messaging

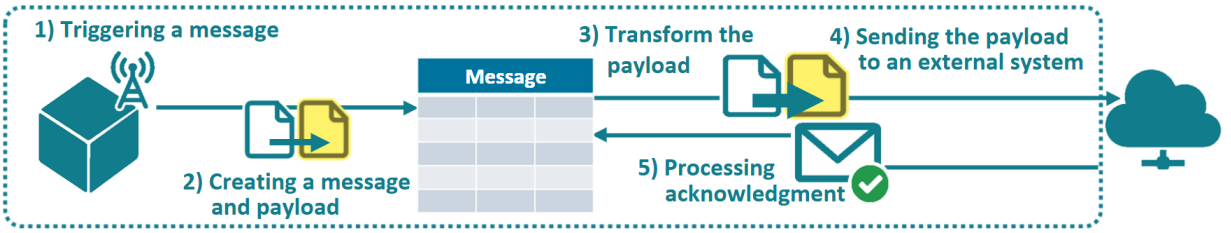


- Guidewire messaging is an asynchronous integration mechanism:
  - Event Based model
    - Predefined and custom events raised by new or changed business data
      - Part of the User transaction
    - Events are evaluated and processed in the Event Fired ruleset
      - Message is created and persisted in the Guidewire database
  - Messages are sent from a Guidewire application to an External system
  - An External system acknowledges receipt and processing of the message (either positively or negatively)

What is Guidewire messaging

The overall architecture of Guidewire messaging is asynchronous. It is possible for an outbound message to be sent to an external system synchronously or asynchronously, depending on the delivery mechanism supported by the external system. However, typically the messages are not processed immediately upon being placed in the message table. Therefore, since the acknowledging message processing aspect is asynchronous, the overall architecture is considered asynchronous.

The messaging stages



1. Triggering a message
   - Newly created or modified entity triggers message creation
2. Creating a message and payload
   - Application creates message and the payload and stores in message table
3. Transform the payload
   - Optional
4. Sending the payload to an external system
   - Application checks message table for messages to process and sends payload
5. Processing acknowledgment
   - External system acknowledges success or reports an error
   - Application processes the received response

Notes

The messaging stages

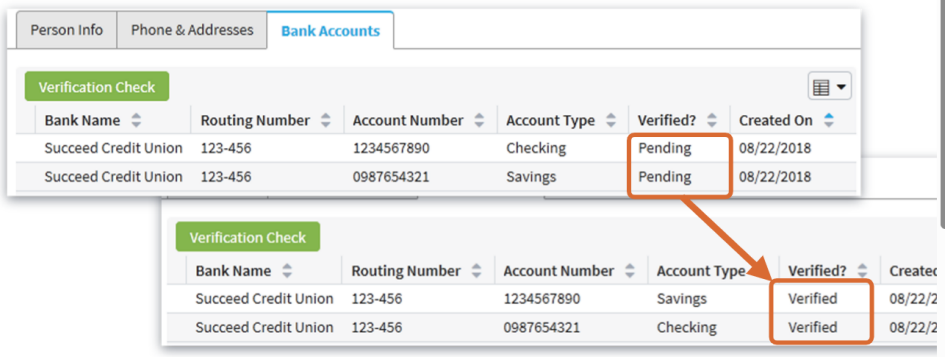
Message creation is triggered by adding an event to an event-aware entity. This almost always involves a data change, though it isn't a requirement.
A positive acknowledgment means that the message was successfully received and processed by the external system.
A negative acknowledgment means that the message was received, but some type of error occurred and the message was not processed. In some cases, the integration code may attempt to resend the message. In other cases, the message must be addressed manually.

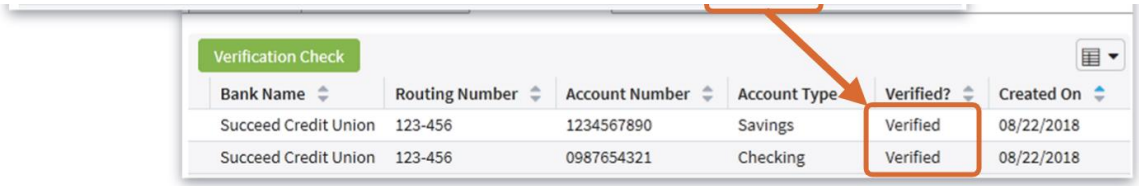
Notes

Use case – Bank account verification

When a new bank account is created, the insurance company wants to verify that the bank name, routing number, and account number reference a real bank account.

Use case – Bank account verification

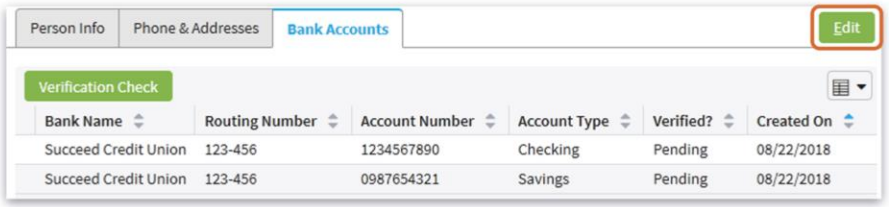




- Messaging process verifies bank account details
  - Bank name, routing number, and account number
  - New and modified accounts
- **Verified?** field shows messaging acknowledgment status
  - Pending, Verified or Invalid

Stage 1 – Triggering a message





- Event
  - A notification of a change (add, update, remove)
  - May trigger Event Fired rule set, if listened for by destination(s)
- BankAccount is an EventAware entity
  - Once a bank account is modified (created, changed, removed) an event is triggered

Notes

Stage 1 – Triggering a message

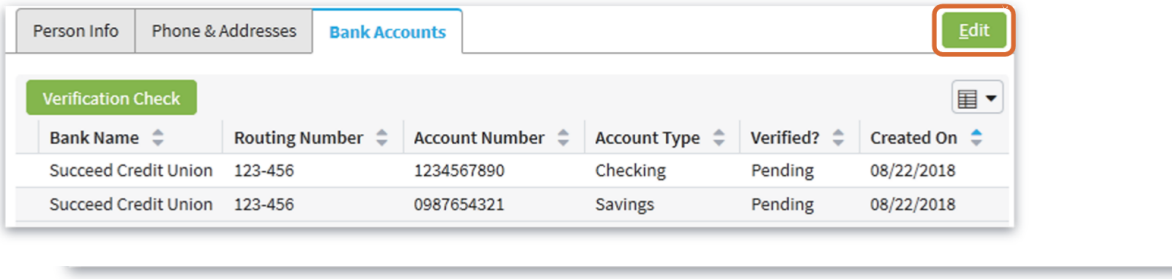
BankAccount is an EventAware entity. The entity specifies an <events/> element in its definition which means that the BankAccount entity delegates to the EventAware delegate. A delegate is an entity associated with an implemented interface that multiple parent entities can reuse. The EventAware entity is an internal entity that implements the messaging interface.

For all entities that implement the EventAware delegate, the Guidewire application automatically creates specific events: entityNameChanged, entityNameRemoved, and entityNameAdded. The BankAccount entity implements the BankAccountChanged, BankAccountRemoved, and BankAccountAdded messaging events.

Stage 1 – Triggering a message

(Continued)





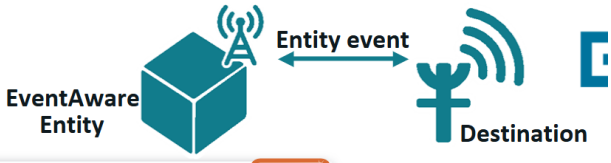
- Messaging destination listens to specific events
  - Subscribes to BankAccountAdded and BankAccountChanged events
  - Represents an external system used to reply to sent messages

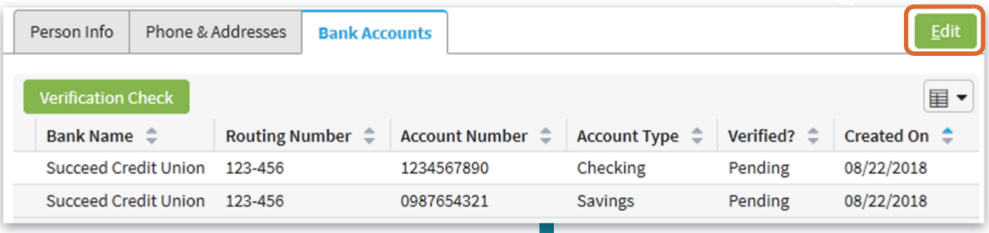
Stage 1 – Triggering a message (Continued)

The destination could be configured to also subscribe to BankAccountRemoved events since the BankAccount entity implements the BankAccountChanged, BankAccountRemoved, and BankAccountAdded messaging events.

Stage 1 – Triggering a message

(Continued)





- Console output details
  - Messaging Destination ID is 13
  - EventAware entity is BankAccount
  - Event is BankAccountChanged

```
*** Stage 1 -- Triggering the message ***
    Event Fired rule set: BAEF1000 - Bank Account Verification
    Messaging Destination ID is 13
    Event Fired rules: BAEF1100 - Bank Account
    EventAware entity is BankAccount
    Event Fired rules: BAEF1110 - Added Changed
    Event is BankAccountChanged
```

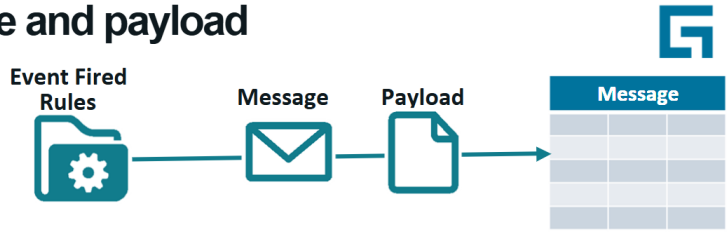
Notes

Stage 1 – Triggering a message (Continued)

When a user clicks Update, each modified BankAccount row in the ListView triggers an event specific to each bank account entity instance. Newly added bank accounts trigger the BankAccountAdded event. Modified bank accounts trigger the BankAccountChanged event. Removed bank accounts trigger the BankAccountRemoved event.

A messaging destination identifies (subscribes to) events of interest, as well as represents an integration channel of an external system, which is used by Guidewire to send messages out and to receive messages back. Bank Account Verification is a messaging destination that subscribes to the BankAccountAdded and BankAccountChanged events. In other words, the Bank Account Verification messaging destination listens for when user creates a bank account or when a user edits a bank account.

Stage 2 – Creating a message and payload



- Event Fired rule set
  - Called for each destination-subscribed messaging event
  - Generates a new message and payload

```
*** Stage 2 -- Creating the message ***
    Event Fired rules: BAEF1111 - Field Changed
    Sending payload to console:
contact,ab:20685
bankName,Succeed Savings and Loan
routingNumber,123-456
accountNumber,45671234
accountType,savings
ageOfMessage,<@@ageOfMessageInSeconds@@>
```

- Message payload
  - Formatted string, such as text, XML or JSON
- Message database table
  - Hub of messaging infrastructure that provides persistent storage for messages with the following status:
    - Unsent (new)
    - Sent and awaiting acknowledgement (pending)
    - Retryable Error (errored)

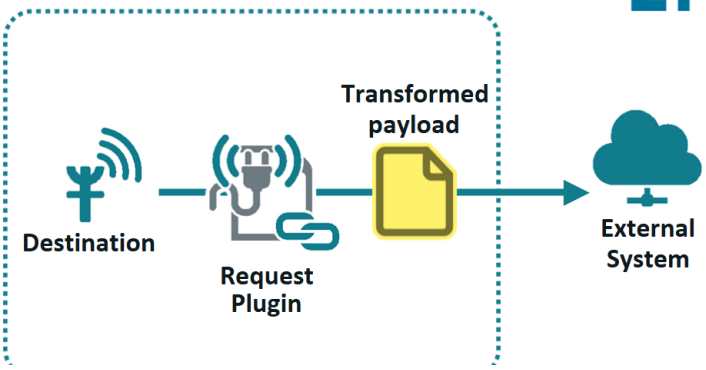
Stage 2 – Creating a message and payload

When the application triggers a subscribed-to messaging event (listened for by a destination), the application calls the Event Fired rule set. An event fired rule set defines a specific root entity as the entity that implements the event messaging interface. The root entity differs from the Message entity. The distinction is important to understand. The root entity represents the instance of the entity that fired the event, in this case, an instance of a BankAccount. The message represents an instance of the Message entity.

All instances of a message entity are stored in the message table. The Message entity has many fields. The field that stores the event-related content of the message is the Payload field. The payload field is what the application sends to the messaging destination subscribing to such a particular event. Payload generation is not reflected in the user interface.

Stage 3 – Transform the payload

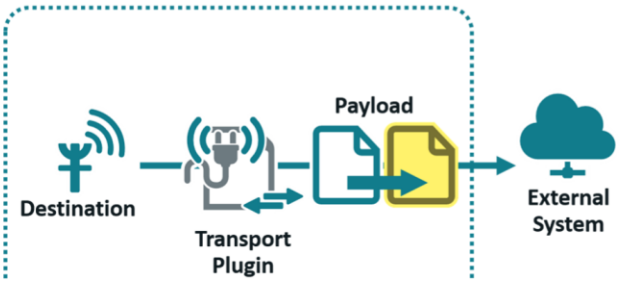
- Destination-specific messaging plugins process the outbound request messages
  - Request plugin
    - Optional payload transformation




```
*** Stage 3 -- Optionally transform the message payload ***
    Request plugin: BankAccountVerificationRequest -- beforeSend()
    Sending transformed payload to console:
contact,ab:20685
bankName,Succeed Savings and Loan
routingNumber,123-456
accountNumber,45671234
accountType,savings
ageOfMessage,13
senderRefID,ab:1203
```

Stage 4 – Sending message to an external system

- Destination-specific messaging plugins process the outbound request messages
  - Transport plugin
    - Sends the payload to the external system
    - Either original payload or transformed payload




```
    *** Stage 4 -- Sending the message ***
    Transport plugin: BankAccountVerificationTransport -- send()
    The Request Plugin transformed the payload
    Sending transformed payload to console:
contact,ab:20685
bankName,Succeed Savings and Loan
routingNumber,123-456
accountNumber,45671234
accountType,savings
ageOfMessage,4
senderRefID,ab:1207
```

Notes

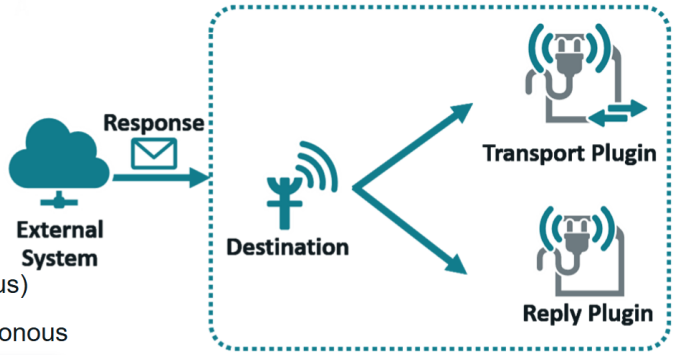
Stage 4 – Sending message to an external system

If you create or change a bank account in TrainingApp and watch the console, you will see a short lag between the click of the Update button and the printing of the payload. This lag occurs because the message table is scanned for new messages only once every few seconds (polling frequency). In other words, the lag occurs because all message integration points are at least partially asynchronous.

A **late binding value** is a message payload value that cannot be determined until right before the message is sent (send-time-only available data v. event-time data).
Payload transformation is when the original payload is altered prior to sending the payload to the external system. Late binding is a form of payload transformation.

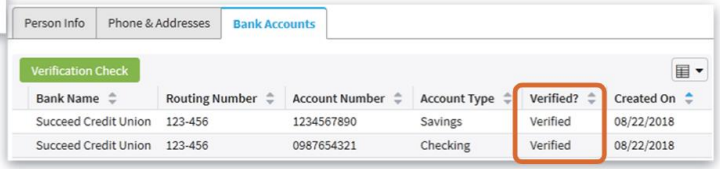
Stage 5 – Processing acknowledgement

- External system
  - Receives sent message
  - Sends back a response
- Guidewire application receives a reply message
  - Processed by Transport plugin (synchronous)
  - Processed by Reply plugin or API (asynchronous




```
*** Stage 5 -- Message acknowledgement ***
Transport plugin: BankAccountVerificationTransport -- send()
Message acknowledged synchronously
Verification status for IsVerified field is Verified
```

- Response determines the **Verified?** field value
  - Verified or Invalid



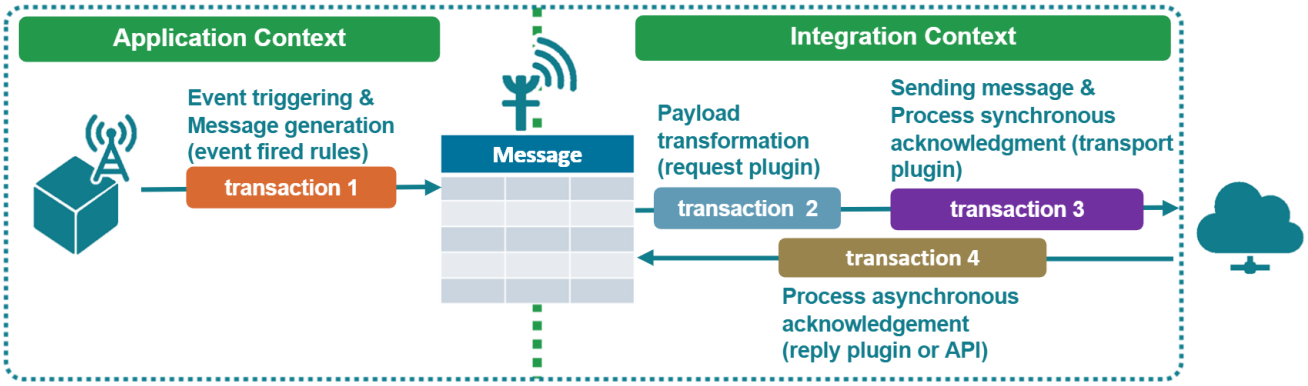
Stage 5 – Processing acknowledgement

In TrainingApp, for contacts with newly added or changed bank accounts, you can see the results of message acknowledgement in the user interface when the **Verified?** value changes from Pending to Verified or Invalid.

A response is a reply from an external system that usually contains some additional data for the Guidewire application to process. Such a reply message may be received and handled by either the transport plugin or the reply plugin, or even a published web service API, depending on the reply message exchange protocol utilized.

Guidewire servers and messaging transactions

A



- Messaging is managed in multiple contexts
  - The application context processes message creation (on server(s) with the “ui” role)
  - All other work is done in the integration context (on server(s) with the “messaging” role)
- Messaging occurs across multiple transactions
  - If an exception is thrown, work in the current transaction is rolled back

Guidewire Proprietary & Confidential – DO NOT DISTRIBUTE

Guidewire servers and messaging transactions

The servers involved at each point of the process are important because they determine where payload work should occur. The majority of payload generation should be done in Event Fired rules on the ui role server that manages the given user's session. However, if some portion of payload generation must be done immediately before the message is sent, that portion of the payload can be generated on the messaging role server.

A message is always processed in at least two and as many as four transactions:
Transaction 1 – Creates the message and generates the payload. This transaction always occurs.
Transaction 2 – Prior to sending the message, an optional transaction may transform the message payload. This is usually done when the message must include information that can only be known immediately before sending the message.
Transaction 3 – The sending of the message to the external system occurs in its own transaction. This transaction always occurs. If the external system acknowledges the message synchronously, then the acknowledgement is included in this transaction.
Transaction 4 – If the external system acknowledges the message asynchronously, then a separate transaction

Message plugin interfaces

```
package gw.plugin.messaging;
...
public interface MessagePlugin extends InternalPlugin {
    void shutdown();
    void suspend();
    void resume() throws InitializationException;
    void setDestinationID(int var1);
}
    public interface MessageBeforeSend extends MessagePlugin {
        String beforeSend(Message var1) throws Exception;
    }
    public interface MessageAfterSend extends MessagePlugin {
        void afterSend(Message var1) throws Exception;
    }
        public interface MessageRequest extends MessageAfterSend, MessageBeforeSend {
        }
        public interface MessageTransport extends MessagePlugin {
            void send(Message var1, String var2) throws Exception;
        }
        public interface MessageReply extends MessagePlugin {
            void initTools(PluginCallbackHandler var1, MessageFinder var2);
        }
```

A

Message plugin interfaces

gw.plugin.messaging.MessagePlugin – the base message plugin interface extended by all specialized message plugin interfaces

shutdown, suspend, resume, setDestinationID methods
These four methods are declared in the common MessagePlugin interface, which all specialized messaging plugin interfaces extend further more. In other words, every messaging plugin class must implement these either directly (or indirectly, e.g. inherited from a common messaging plugin implementation base class).

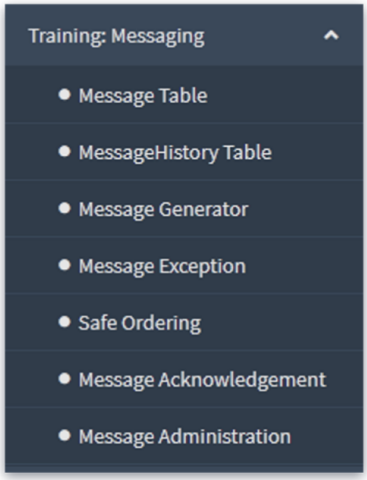
The first three methods (shutdown/suspend/resume) provide the essential control over a messaging plugin instance status (and ultimately a messaging destination), and are consumed either implicitly by the GW messaging framework when/as needed (app shutdown, failed send, unhandled exception, app startup) or explicitly by user actions via the messaging admin UI (suspend/resume).

The setDestinationID method is called by the GW application framework at the time a messaging plugin class is loaded/instantiated for a particular destination. This allows the initialized plugin class instance to preserve and later utilize/reference the actual destination ID for which this messaging plugin instance has been loaded (such as SenderRefID generating in the request plugin, or the reply plugin message finder API, logging in all of them etc.).

Message Administration

Training: Messaging

- Available only in the TrainingApp application
  - **Administration tab → Training: Messaging**
- Message Table
  - Shows pending send (unsent), pending acknowledgement (sent), and Retryable error (errored messages)
- MessageHistory Table
  - Shows acknowledged, duplicate, or errored messages
- Message Administration
  - A copy of the Message Queues screen found in all InsuranceSuite applications

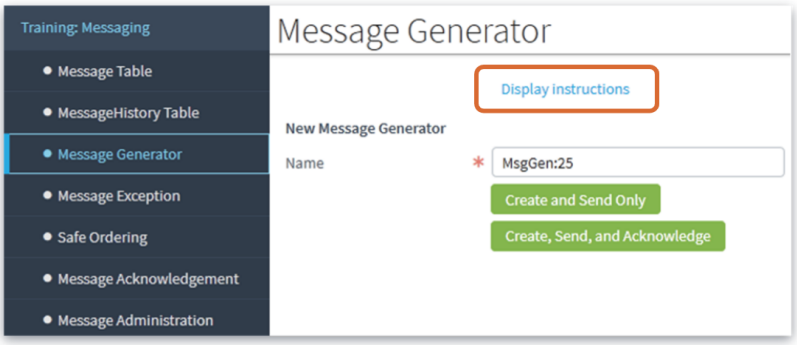


Training: Messaging

The screens in the **Training: Messaging** location group (including the Message Table screen shown above) are unique to the TrainingApp only and have been created for learning purposes only. The genuine Guidewire applications provide a **Message Queues** administration screen (under the Monitoring location group).

Training: Messaging – Demo screens

- Each demo screen has a **Display instructions** link
- Message Generator
  - Creates a new message for a given message reference ID
- Message Exception
  - Throws an exception for a given message reference ID



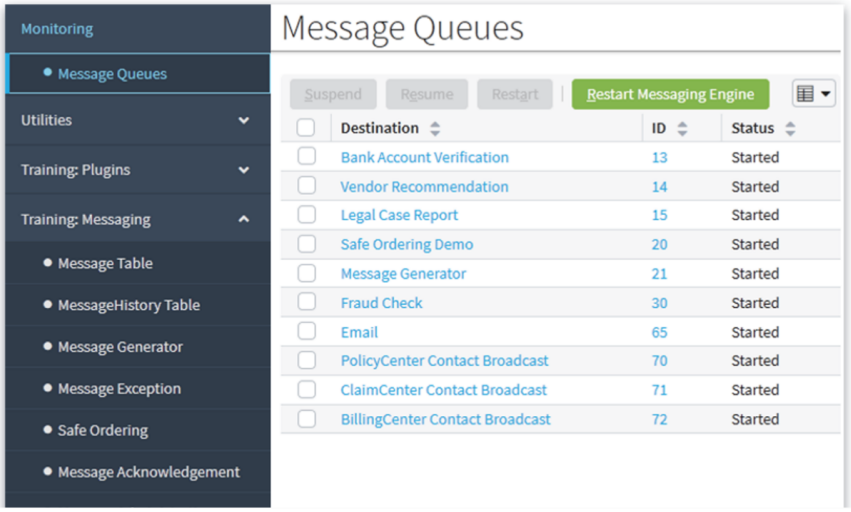
- Message Acknowledgment
  - Simulates message acknowledgment from an external system for a given message reference ID
- Safe Ordering
  - Lets you send multiple messages for the same entity for safe-order and non-safe-order entity

Training: Messaging – Demo screens

The screens in the **Training: Messaging** location group (including the Message Table screen shown above) are unique to the TrainingApp only and have been created for learning purposes only. The genuine Guidewire applications provide a **Message Queues** administration screen (under the Monitoring location group).

Message Queues

- A series of drilldown screens for message administration
- Administration tab
  - **Monitoring → Message Queues**, or
  - **Training: Messaging → Message Administration**
- Suspend, resume, and view messages for each destination



Notes

Message Queues

The Message Queues screen is available in all InsuranceSuite applications. The screens in the **Training: Messaging** location group (including the Message Table screen shown above) are unique to the TrainingApp only and have been created for learning purposes only.
From Message Queues, you can suspend, resume, and view messages for each destination

The Message Queues screens group all unacknowledged messages by:

- Destination, then...
- Primary entity (safe-ordering, plus one set for all unordered), then...
- Status: failed, retryable error, in flight, and unsent

Question and Answer

1) **What are the five stages of Guidewire messaging?**

1. Triggering a message.
2. Creating a message and payload.
3. Optionally transform the payload.
4. Sending the message to an external system.
5. Processing message acknowledgment.

2) **What are the three possible statuses for messages that are in the message table?**

1. Pending send.
2. Pending acknowledged (sent but no acknowledgement received).
3. Retryable error (sent and an error reported by external system, a negative acknowledgement or NACK).

3) **For a Guidewire implementation using clustering, what two server roles execute the work for messaging? What work is done by each server role?**

The two clustering server roles are: the **ui** role and the **messaging** role.